A Comparative Study of Deep Q-Network Variants Across Diverse Gymnasium Environments

Samiya Ali Zaidi
School of Science and Engineering
Habib University
Karachi, Pakistan
sa07171@st.habib.edu.pk

Javeria Azfar
School of Science and Engineering
Habib University
Karachi, Pakistan
ja07622@st.habib.edu.pk

Huzaifah Tariq Ahmed
School of Science and Engineering
Habib University
Karachi, Pakistan
ha07151@st.habib.edu.pk

***Abstract*—Deep Q-Networks (DQNs) have played a pivotal role in advancing deep reinforcement learning by enabling agents to learn optimal policies directly from high-dimensional inputs. While the original DQN framework demonstrated remarkable success, it suffers from drawbacks such as value overestimation and training instability. This project presents a systematic comparison of three prominent DQN variants—Double DQN, Dueling DQN, and Elastic Step DQN (ES-DQN)—across diverse Gymnasium environments, including CartPole-v1 and LunarLander-v2. The study evaluates each algorithm's sample efficiency, convergence stability, and value estimation behavior under standardized conditions. Through reproducible experimentation and rigorous evaluation, the project aims to identify environment-specific strengths and trade-offs, providing actionable insights for the selection and deployment of DQN-based algorithms in resource-constrained or task-specific reinforcement learning applications.**

***Index Terms*—Deep Reinforcement Learning, Deep Q-Networks, Double DQN, Dueling DQN, Elastic Step DQN, OpenAI Gym**

I. Introduction

Reinforcement Learning (RL) has emerged as a powerful paradigm for training autonomous agents to make sequential decisions through interaction with dynamic environments. A significant breakthrough in this domain was the introduction of Deep Q-Networks (DQNs) by Mnih et al. [1], which enabled agents to learn directly from high-dimensional sensory inputs by combining Q-learning with deep neural networks. This achievement set a new benchmark in value-based decision-making, particularly in discrete action spaces.

Despite its success, the original DQN algorithm suffers from overestimation bias, inefficient exploration, and training instability. To address these issues, a range of DQN variants have been proposed. Double DQN [2] reduces value overestimation by decoupling action selection and evaluation. Dueling DQN [3] introduces separate estimations of state value and action advantages for improved learning efficiency. More recently, Elastic Step DQN (ES-DQN) [4] incorporates a dynamic update mechanism to adapt learning horizons based on state similarity, aiming to enhance convergence and reduce sensitivity to hyperparameters.

While these algorithms have been studied individually, little work offers a unified, empirical comparison across diverse environments under standardized conditions. Most benchmarks focus on high-compute Atari games, offering limited insights for practical, resource-constrained applications. This research fills that gap by comparing three widely used DQN variants, Double DQN, Dueling DQN, and ES-DQN, across three Gymnasium environments: CartPole-v1, LunarLander-v3, and FrozenLake-v1. These environments differ in complexity, reward sparsity, and transition dynamics, making them suitable for evaluating the generalizability and efficiency of learning algorithms.

This paper aims to evaluate each algorithm's training sample efficiency, stability, and Q-value estimation trends under consistent experimental settings. The goal is to identify when and why specific DQN variants perform better, and to provide actionable insights for both academic and applied reinforcement learning research.

The remainder of this paper is structured as follows: Section II surveys related work and motivates the study. Section III details the experimental methodology, including network design and environment setup. Section IV presents empirical results, followed by critical analysis in Section V. Section VI concludes the paper, while Section VII discusses limitations and future directions.

II. Literature Review

In the exploration of Deep Q-Network (DQN) variants applied to Gymnasium environments, [5] offer a valuable, though tangential, perspective by examining the integration of Double Q-learning principles within the Asynchronous Advantage Actor-Critic (A3C) [6] framework. The paper proposes two architectures, Double A3C and Less Shared (LS) Double A3C, that aim to reduce overestimation bias by maintaining dual value function estimates (V1 and V2), updating them alternately during training. Although not directly based on DQN, the study aligns with the broader objectives of DQN

variants by tackling overestimation and enhancing learning stability. Empirical evaluations across Gymnasium-based Atari games (Breakout, Ice Hockey, and Pong) reveal that while these A3C variants often outperform traditional DQN in terms of average reward and data efficiency, they do not consistently outperform the baseline A3C model, suggesting limited benefit from the added architectural complexity. Particularly in LS Double A3C, reduced parameter sharing leads to potentially noisier updates, and the performance varied significantly by game, highlighting the environment-specific nature of these algorithms. Additionally, the paper emphasizes hardware considerations, such as increased GPU use in asynchronous models versus higher memory requirements in DQN due to experience replay, and calls attention to the trade-offs in resource-constrained scenarios. Despite its limited focus on DQN directly, the paper reflects key trends in deep reinforcement learning which are, enhancing training stability via architectural innovation, analyzing algorithm performance across diverse environments, and improving model interpretability through tools like saliency maps. It also underscores the growing challenge of generalization across Gymnasium environments and the importance of tailoring algorithms to game-specific dynamics. These insights highlight gaps in understanding how architectural changes impact stability and overestimation in asynchronous learning settings, which can inform future comparisons between synchronous and asynchronous methods or inspire hybrid approaches that balance sample efficiency with stable value estimation.

[7] proposes a simple yet effective approach to multi-agent reinforcement learning (MARL), where each agent is a Deep Q-Network (DQN) responsible for a binary action, either to act or not, while sharing a common state and reward signal. This architecture, referred to as Class-Specific DQN (CS-DQN) and its Double DQN variant (CS-DDQN), simplifies the coordination process by removing the need for explicit inter-agent communication and allows the use of shared experience replay buffers. The authors demonstrate the method's effectiveness on CartPole-v1, LunarLander-v2, and a custom Maze Traversal environment, showing faster convergence and better performance than traditional single-agent DQN/DDQN baselines. A key strength of the approach lies in its simplicity, extensibility to any DQN variant, and resource efficiency. However, its reliance on binary action decomposition and the absence of comparisons with other MARL methods limits its generalizability and broader impact assessment. Moreover, while the shared reward and state structure streamlines learning, it may inhibit the emergence of specialized agent behaviors. The authors identify future work such as introducing inter-agent communication and testing on more complex environments, which could enhance scalability and performance. Overall, this paper contributes a lightweight MARL strategy that performs well in discrete action spaces and aligns with broader trends in reducing complexity in deep reinforcement learning systems.

Exploration remains a central challenge in deep reinforcement learning (DRL), particularly in high-dimensional environments where random exploration strategies are inefficient. Addressing this, Zhang et al. [8] proposed the Noisy Attention Exploration Module (NAEM), a lightweight and generalizable neural architecture designed to enhance exploration through a combination of learnable noise and attention mechanisms. NAEM consists of two parallel sub-modules: a channel noisy attention component, which injects state-dependent Gaussian noise into the attention weights to encourage adaptive exploration, and a spatial attention component that preserves salient spatial features using a CBAM-like mechanism. This design enables effective exploration without disrupting task-relevant feature representations. The authors integrated NAEM into two prominent DRL algorithms, Rainbow (a value-based method) and SAC-Discrete (an actor-critic method) demonstrating broad compatibility and significant benefits. NAEM achieved more than 130% performance improvement over baseline methods (e.g., NoisyNet-DQN, NoisyNet-A3C) on most Atari 2600 games, while also reducing parameters by 44% and speeding up training by approximately 30%. Notably, NAEM offers a novel balance between structured feature extraction and stochastic exploration, outperforming conventional methods such as random action selection or fixed noisy layers. Despite its strengths, limitations include its limited attention to temporal dependencies, which may hinder performance in partially observable or sequential tasks. The authors suggest future work in extending NAEM to recurrent architectures, multi-agent settings, and real-world applications such as robotics, as well as enhancing its interpretability in decision making processes [8].

[4] introduces Elastic Step DQN (ES-DQN), a novel approach that dynamically adjusts the multi-step update horizon in Deep Q-Networks (DQNs) based on state similarity. Instead of using a fixed n-step parameter, ES-DQN leverages unsupervised clustering, specifically HDBSCAN applied to hidden-layer outputs, to group similar states and determine the appropriate update horizon. The method employs a state memory bank to capture diverse representations and refines feature extraction by using hidden node outputs rather than raw states. Empirical results on Cartpole, Mountain Car, and Acrobot demonstrate that ES-DQN achieves competitive or superior performance compared to single-step, fixed n-step, Double DQN, and Average DQN variants, particularly in reducing the overestimation of Q-values. While the algorithm shows slower convergence during early learning phases and its reliance on clustering may limit scalability to high-dimensional tasks, its dynamic update mechanism effectively alleviates hyper-parameter sensitivity. The study suggests that future work could explore hybrid approaches, alternative similarity measures, and applications to more complex environments, aligning with broader trends in adaptive and resource-efficient reinforcement learning systems.

[9] presents a quantum-enhanced deep Q-learning framework that replaces conventional neural networks with parametrized quantum circuits (PQCs) to approximate Q-values as expectation values. The study adapts the DQN architecture by integrating hardware-efficient PQCs and systemati-

cally investigates various data encoding and readout strategies, such as data re-uploading and trainable input weights, through detailed ablation studies. The results on benchmarks like Cart Pole show that, while PQC-based agents can achieve competitive performance with fewer parameters, they are notably more sensitive to architectural choices and experience training instabilities as model complexity increases. These findings underline that the success of quantum Q-learning hinges more on careful observable selection and hyperparameter tuning than on mere scaling of parameters. The authors suggest future research directions, including improving training stability, developing hybrid quantum–classical architectures, and expanding evaluations to more complex environments, thereby contributing valuable insights toward realizing quantum advantage in reinforcement learning tasks.

III. Methodology and Experimental Documentation

In this section, we outline the methodology employed to investigate the research question posed in this study. The methodology is organized into four subsections, each corresponding to a key component of our experimental design: (A) Double Deep Q-Network (Double-DQN), (B) Dueling Deep Q-Network (Dueling DQN), (C) Elastic Step Deep Q-Network (ES-DQN), and (D) Environment-Specific Setup, which details the environments used and their configurations.

A. Double DQN

Mitigates overestimation by using separate networks for selection and evaluation. All agents use a shared feedforward Q-network.

1) Network Architecture and Training Setup:

- **Input Layer:** Matches the dimensionality of the environment's state space (CartPole: 4, LunarLander: 8, FrozenLake: 16 one-hot).
- **Hidden Layers:** Two fully connected layers, each with 128 ReLU units.
- **Output Layer:** One linear layer mapping to the number of discrete actions (e.g., 2 for CartPole, 4 for LunarLander).
- **Q-Value is computed by:**

$$Q(s,a) \leftarrow r+$$

$$\gamma \cdot Q_{\text{target}}\left(s', \arg\max_{a'} Q_{\text{online}}(s',a')\right) \cdot (1-\text{done})$$

2) Experience Relay and Exploration: The agent leverages a uniform experience replay buffer with the following setup:

- **Replay Buffer:**
  - Size: 50,000 for CartPole/LunarLander; 10,000 for FrozenLake.
  - Batch Size: 64 (CartPole/LunarLander), 32 (FrozenLake).
- **Exploration:** An exponentially decaying ε-greedy policy is used:

$$\varepsilon_t = \varepsilon_{\text{end}} + (\varepsilon_{\text{start}} - \varepsilon_{\text{end}}) \cdot \exp\left(-\frac{t}{\varepsilon_{\text{decay}}}\right)$$

  - $\varepsilon_{\text{start}} = 1.0$, $\varepsilon_{\text{end}} = 0.01$
  - Decay: 10,000 steps for CartPole/LunarLander, 500 for FrozenLake.

3) Hyperparameter Selection: Key training parameters are listed in Table I. CartPole and LunarLander share similar tuning, whereas FrozenLake uses faster updates and more aggressive exploration due to its sparse reward structure. The number of episodes used for each environment was:

- **FrozenLake-v1**: 500
- **Cartpole-v1**: 500
- **LunarLander-v3**: 500

TABLE I
HYPERPARAMETERS USED FOR DOUBLE DQN

Hyperparameter	Value	Reason
`learning_rate`	$1e^{-3}$	Stable convergence with Adam across all environments
`gamma`	0.99	Prioritizes long-term reward accumulation
`batch_size`	64 / 32	Larger batch for stability (CartPole, LunarLander); smaller for FrozenLake
`buffer_size`	50000 / 10000	Larger buffer captures more transitions in long episodes
`tau (soft update)`	$1e^{-3}$ / $1e^{-2}$	Faster update in FrozenLake to handle sparse rewards
`epsilon_start/end`	$1.0 \rightarrow 0.01$	Gradual decay encourages initial exploration then policy refinement

4) Environment-Specific Setup::

- **CartPole-v1:** Low-dimensional (4D), dense reward environment. Converges rapidly in $\sim$300 episodes with moderate overestimation bias.
- **LunarLander-v3:** Higher-dimensional (8D) with sparse and noisy rewards. Requires more training episodes and shows higher Q-value overestimation.
- **FrozenLake-v1:** Discrete 4x4 grid with binary rewards and stochastic transitions. Learning is slow, and sparse rewards require tuned exploration decay and faster soft-updates.

B. Duelling DQN

Duelling DQN uses two streams to independently estimate the value and advantage functions. We use a feedforward architecture with the following configuration:

1) Network Architecture:

- **Input Layer:** Takes in environment specific state vectors (e.g 4D for CarPole-V1, 8D for LunarLander-V3 and 16D one-hot for FrozenLake-V1).
- **Hidden Layer:** A shared fully connected layer with 128 ReLU units.
- **Duelling Streams:**
  - **Value Stream:** Two layers ($128 \rightarrow 128 \rightarrow 1$) estimate the state-value $V(s)$.

  - **Advantage Stream:** Two layers ($128 \rightarrow 128 \rightarrow$ action size) estimate the advantage $A(s, a)$.
- The final computed Q-Values are:

$$Q(s,a) \leftarrow V(s) + \left(A(s,a) - \frac{1}{|A|} \sum_{a'} A(s,a') \right)$$

2) Experience Relay and Exploration: A fixed-capacity replay buffer (size = 10,000) stores transitions (s, a, r, s', d), allowing for decorrelated minibatch training. An epsilon-greedy strategy governs exploration:

- ϵ starts at 1.0, decays to 0.01 with a decay rate of 0.995.
- Ensures high initial exploration with gradual exploitation of learned policies.

3) Hyperparameter Selection: The number of episodes used for each environment was:

- **FrozenLake-v1**: 500
- **Cartpole-v1**: 500
- **LunarLander-v3**: 500

Table II provides the details of the hyperparameters used and the rationale behind them.

TABLE II
HYPERPARAMETERS USED FOR DUELING DQN

Hyperparameter	Value	Reason
`learning_rate`	0.001 / 0.0005	Higher for CartPole/FrozenLake; lower for LunarLander to improve stability
`gamma`	0.99	Balances short- and long-term rewards across all environments
`batch_size`	32	Suitable for efficient updates and generalization
`target_update_freq`	10 steps	Stabilizes learning with regular target network updates
`memory_size`	10,000	Ensures diverse experience replay without excessive memory

C. ES-DQN

ES-DQN is a variant of DQN designed to improve learning stability and sample efficiency by modulating the TD update step size dynamically, allowing for more flexible credit assignment over multi-step transitions.

1) Network Architecture: The ES-DQN agent uses a neural network as a function approximator to estimate Q-values as shown below:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) +$$
$$\alpha \left[\sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_{a'} Q(s_{t+n}, a') - Q(s_t, a_t) \right]$$

The network architecture consists of

- a fully connected multilayer perceptron with ReLU activations
- a input layer size corresponds to the dimension of the encoded state
- two hidden layers

2) Experience Relay Buffer: We implemented an N-step replay buffer to store transitions in the form of `(state, action, reward, next_state, done)`. The ES-DQN uses the N-step return to compute the target Q-value, reducing bias from bootstrapping. The epsilon-greedy policy is also used (as shown in Table III) to encourage exploration in the initial episodes.

3) Hyperparameters Used: The number of episodes used for each environment was:

- **FrozenLake-v1**: 1000
- **Cartpole-v1**: 5000
- **LunarLander-v3**: 500

These values were iteratively chosen to ensure that the environment was solved. Table III provides the details of the rest of the hyperparameters used and the rationale behind them.

TABLE III
HYPERPARAMETERS USED FOR ES-DQN

Hyperparameter	Value	Reason
`learning_rate`	$1e^{-3}$	Balanced speed and stability using Adam optimizer
`gamma`	0.9	Suitable for short episodes, prioritizes near-term rewards
`n_step`	3	Effective trade-off between 1-step and Monte Carlo returns
`batch_size`	32	Memory-efficient and statistically reliable for small environments
`buffer_size`	100000	Prevents overfitting, ensures diversity in replayed samples
`epsilon_start/end`	$1.0 \rightarrow 0.01$	Smooth transition from exploration to exploitation

D. Environment-Specific Setup

- **FrozenLake-v1:** Discrete 4x4 grid; sparse rewards; one-hot encoded state input.
- **CartPole-v1:** Continuous 4D state; reward given per timestep survived.
- **LunarLander-v3:** Complex 8D state with continuous rewards; engines and dynamics make control challenging.

IV. RESULTS AND ANALYSIS

We evaluated three Deep Q-Network (DQN) variants—Elastic DQN, Duelling DQN, and Double DQN—across three Gymnasium environments (CartPole, LunarLander, and FrozenLake). Performance was quantified in terms of sample efficiency (AUC%), average reward, stability (standard deviation and coefficient of variation), and over-estimation bias (mean maximum Q-value).

A. Sample Efficiency

Table IV summarizes the normalized area under the learning curve (AUC%) for each algorithm–environment combination.

Duelling DQN converged fastest on CartPole, achieving an AUC of 43.62%, whereas Elastic DQN showed only marginal gains. In LunarLander, Elastic DQN's negative AUC indicates

TABLE IV
SAMPLE EFFICIENCY (AUC%)

Environment	Elastic DQN	Duelling DQN	Double DQN
CartPole	3.51%	43.62%	37.96%
LunarLander	–32.19%	8.87%	3.02%
FrozenLake	47.20%	15.70%	39.50%

unstable early learning, and both Duelling and Double DQN attained only modest positive efficiency. In the sparse–reward FrozenLake environment, Elastic DQN recorded the largest AUC (47.20%), driven by intermittent reward spikes rather than continuous policy improvements (see Figure 1(a)–(c)).

B. Average Reward and Stability

Table V presents average returns and their variability for each algorithm.

TABLE V
AVERAGE REWARD AND STABILITY

Environment	Algorithm	Mean Reward	Std. Dev.	CV
CartPole	Elastic DQN	16.66	11.37	0.68
	Duelling DQN	218.63	172.88	0.79
	Double DQN	76.10	80.20	1.05
LunarLander	Elastic DQN	-64.70	153.83	-2.38
	Duelling DQN	35.69	153.00	4.29
	Double DQN	9.20	176.40	19.20
FrozenLake	Elastic DQN	0.47	0.50	1.06
	Duelling DQN	0.16	0.36	2.31
	Double DQN	0.40	0.49	1.24

Only Duelling DQN approached optimal performance in CartPole with an average reward above 200, but exhibited high variance. Double DQN stabilized after an initial learning phase, achieving moderate performance (mean 76.10). In LunarLander, all variants showed large variances ($\sigma > 150$), and none met the 200-point success threshold consistently. FrozenLake's sparse rewards produced near-zero means and high relative variability across all algorithms (see Figure 2(a)–(c)).

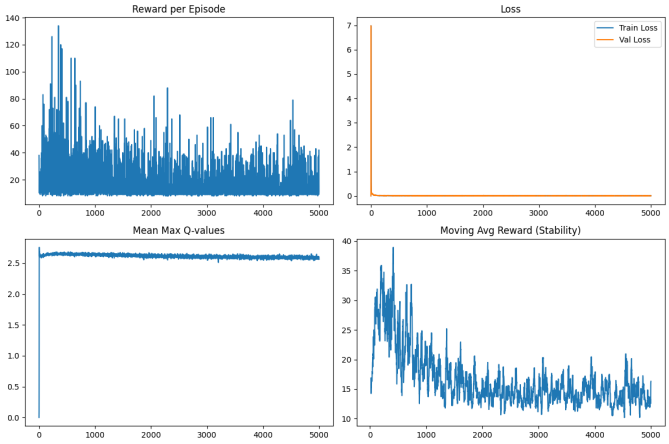
C. Over-Estimation Bias

Table VI compares mean maximum Q-values as an indicator of estimation bias.

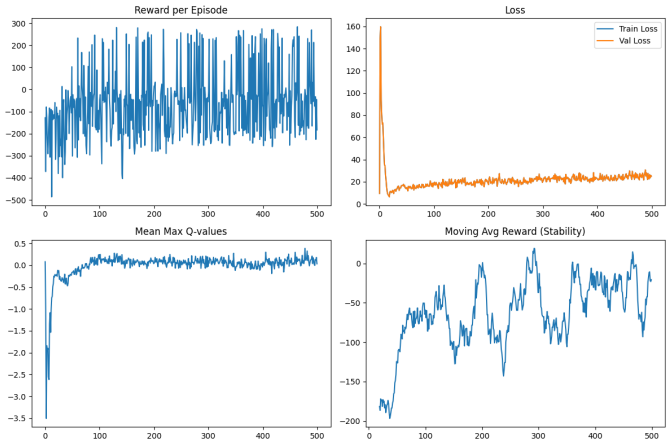
TABLE VI
MEAN MAXIMUM Q-VALUE

Environment	Elastic DQN	Duelling DQN	Double DQN
CartPole	2.61	404.69	16.30
LunarLander	-0.01	56.97	19.90
FrozenLake	0.13	0.41	0.37

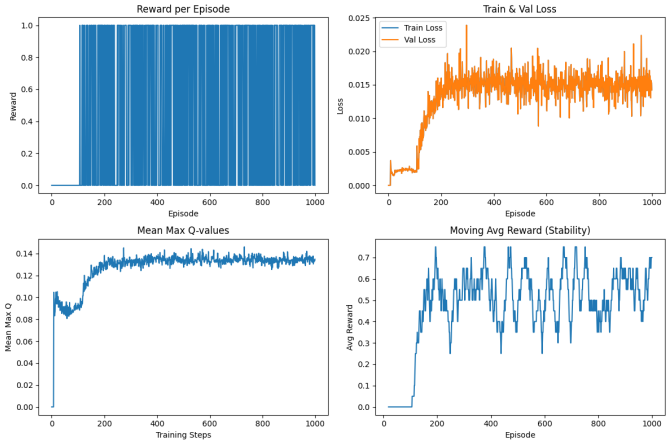
Duelling DQN exhibited significant over–estimation on CartPole with Q-values averaging 404.69, which aligns with its high return variability. Double DQN moderated these values to 16.30, demonstrating effective bias reduction. Elastic DQN remained conservative in its estimates across environments (see Figure 3(a)–(c)).



(a) CartPole



(b) LunarLander

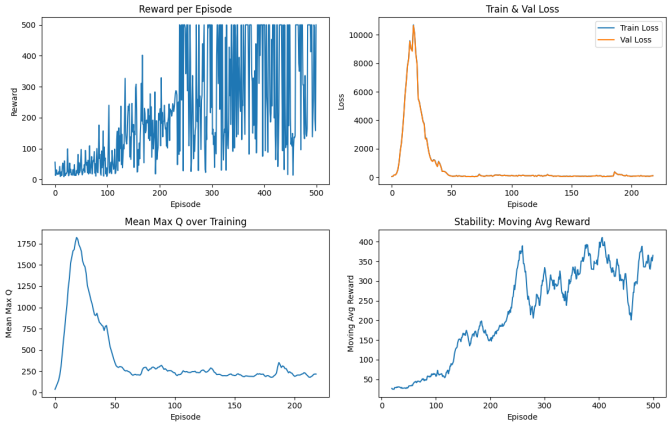


(c) FrozenLake

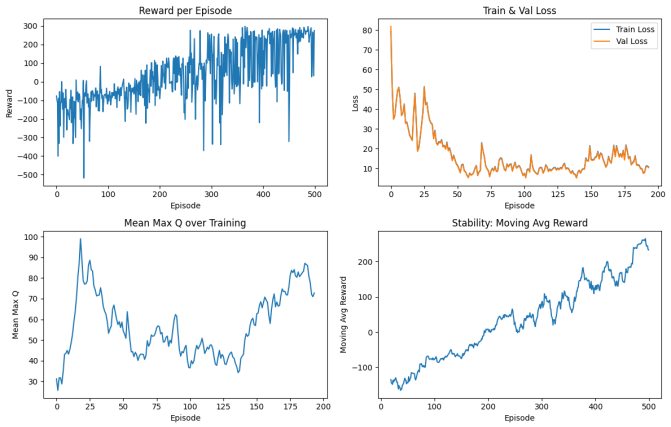
Fig. 1. Elastic DQN results on (a) CartPole, (b) LunarLander, and (c) FrozenLake. Each subfigure shows reward per episode, training & validation loss, mean max Q-values, and moving average reward.

D. Findings

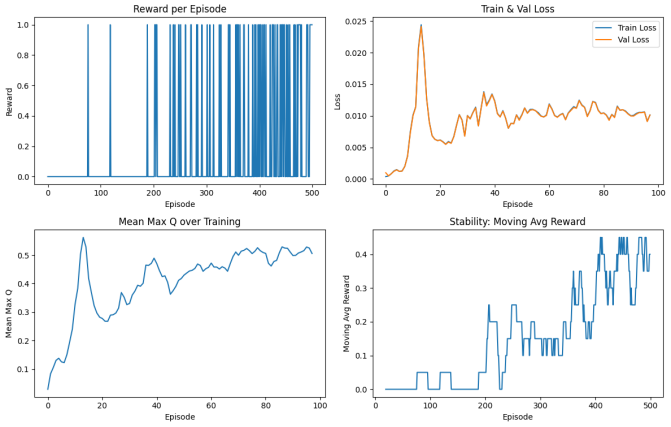
The comparative evaluation reveals that algorithmic enhancements yield environment-dependent benefits. Duelling DQN's separation of state-value and advantage streams leads to rapid learning in dense-reward, low-variance settings such as CartPole, but introduces pronounced over-estimation and



(a) CartPole



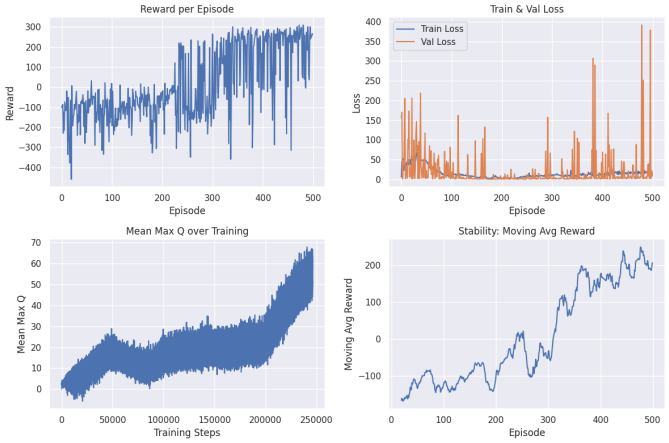
(b) LunarLander



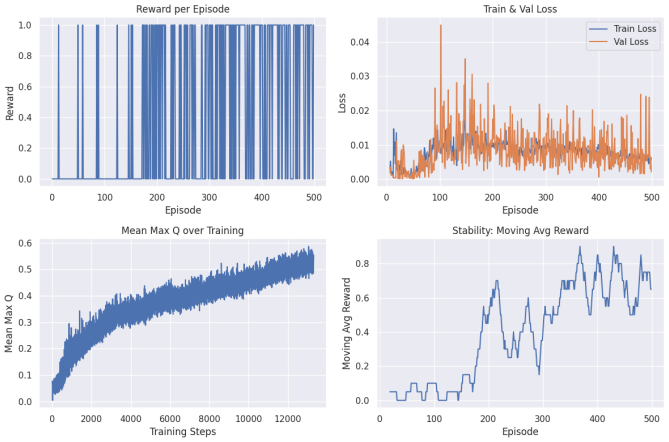
(c) FrozenLake

Fig. 2. Duelling DQN results on (a) CartPole, (b) LunarLander, and (c) FrozenLake. Each subfigure shows reward per episode, training & validation loss, mean max Q-values, and moving average reward.

(a) CartPole



(b) LunarLander



(c) FrozenLake

Fig. 3. Double DQN results on (a) CartPole, (b) LunarLander, and (c) FrozenLake. Each subfigure shows reward per episode, training & validation loss, mean max Q-values, and moving average reward.

higher return volatility. Conversely, Double DQN's bias-correction mechanism produces more calibrated value estimates, resulting in smoother learning trajectories at the expense of slower convergence. Elastic DQN's adaptive update approach can leverage rare success events in sparse-reward environments—e.g., FrozenLake—but fails to deliver consistent policy improvements in more complex domains.

These findings suggest that the selection of a DQN variant should be informed by the reward structure and stochastic properties of the target environment. For tasks characterized by dense and stable reward signals, Duelling DQN offers the fastest sample efficiency. In contrast, for environments with sparse or noisy rewards, Double DQN's conservative updates and reduced over-estimation bias provide more reliable and

stable convergence. Elastic DQN may be suitable when occasional high-value transitions are expected, but its inconsistent stability warrants caution and further hyperparameter tuning.

V. Conclusion

This study presents a rigorous comparative evaluation of Elastic Step DQN (ES-DQN), Duelling DQN, and Double DQN across environments with varying reward densities and stochasticity. Our key contribution lies in establishing clear performance regimes for each variant: Duelling DQN demonstrates superior sample efficiency in dense-reward settings like CartPole-v1 due to its fast action-value differentiation; Double DQN consistently offers the most stable convergence and accurate value estimates across all tasks by mitigating overestimation bias; and ES-DQN, with its adaptive multi-step updates, outperforms in sparse-reward environments such as FrozenLake-v1, highlighting its strength in long-horizon credit assignment.

We also show that while Duelling DQN accelerates early learning, it exhibits volatility due to overconfident Q-value predictions. Double DQN's decoupled evaluation yields smoother, more reliable learning curves, while ES-DQN's performance is highly sensitive to hyperparameter tuning despite its theoretical advantages. Importantly, our work underscores that algorithm effectiveness is task-dependent and no single variant dominates universally.

By contextualizing each algorithm's strengths and limitations, this research provides actionable guidance for algorithm selection based on environment characteristics, marking a step toward more principled deep RL deployment.

VI. Limitations and Future Directions

While our comparative evaluation highlights key strengths and trade-offs among Double DQN, Dueling DQN, and ES-DQN, it is based on a limited set of benchmark tasks and fixed hyperparameter settings. Extending the analysis to additional, more complex environments and employing systematic, automated tuning could further refine our understanding of each variant's robustness. Incorporating complementary techniques, such as prioritized replay or distributional value estimation, and evaluating across multiple random seeds would also help confirm the consistency of our findings. Finally, a deeper theoretical investigation into each algorithm's update mechanism may offer principled guidance for parameter selection and inspire new hybrid approaches that leverage the complementary advantages highlighted by our study.

References

[1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2 2015.

[2] Hado Van Hasselt, Arthur Guez, and David Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the AAAI conference on artificial intelligence*, 2016, vol. 30.

[3] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas, "Dueling network architectures for deep reinforcement learning," in *International conference on machine learning*. PMLR, 2016, pp. 1995–2003.

[4] Adrian Ly, Richard Dazeley, Peter Vamplew, Francisco Cruz, and Sunil Aryal, "Elastic step dqn: A novel multi-step algorithm to alleviate overestimation in deep q-networks," *Neurocomputing*, vol. 576, pp. 127170, 2024.

[5] Yangxin Zhong, Jiajie He, and Lingjie Kong, "Double a3c: Deep reinforcement learning on openai gym games," *arXiv preprint arXiv:2303.02271*, 2023.

[6] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *International conference on machine learning*. PmLR, 2016, pp. 1928–1937.

[7] Abdul Mueed Hafiz and Ghulam Mohiuddin Bhat, "Deep q-network based multi-agent reinforcement learning with binary action agents," *arXiv preprint arXiv:2008.04109*, 2020.

[8] Zhenwen Cai, Feifei Lee, Chunyan Hu, Koji Kotani, and Qiu Chen, "NAEM: Noisy Attention Exploration Module for Deep Reinforcement Learning," *IEEE Access*, vol. 9, pp. 154600–154611, 1 2021.

[9] Andrea Skolik, Sofiene Jerbi, and Vedran Dunjko, "Quantum agents in the gym: a variational quantum algorithm for deep q-learning," *Quantum*, vol. 6, pp. 720, 2022.